

LECTURE 17

Accessibility: ARIA Intro

Screen Readers · Alt Text · ARIA Roles — Student Study Notes

Contents

What Is Web Accessibility2

1. Screen Readers.....2

2. Alt Text3

3. Basic ARIA Roles.....4

4. Example: Before and After5

5. Key Takeaways.....6

What Is Web Accessibility

Web accessibility means building web pages so that people with different abilities can use them — including users who are blind or have low vision, users who cannot use a mouse and rely on a keyboard, users who are deaf or hard of hearing, and users with cognitive differences.

Accessibility is not a separate feature added at the end of a project — it's a way of writing HTML correctly from the start. A page built with proper structure, meaningful text, and correct tags is accessible by default. A page built carelessly needs extra work later to fix.

Three Building Blocks

- Screen readers — how assistive tech reads a page
- Alt text — text alternatives for images
- ARIA roles — attributes that describe custom components to assistive tech

1. Screen Readers

What a screen reader is

A screen reader is software that converts on-screen content into speech or braille output. It's used mainly by people who are blind or have low vision. Common screen readers:

Screen Reader	Cost	Platform
NVDA	Free	Windows
JAWS	Paid	Windows
VoiceOver	Built-in	macOS / iOS
TalkBack	Built-in	Android

How a screen reader reads a page

A screen reader does not “see” a webpage the way a sighted user does — it doesn't care about colors, fonts, or spacing. Instead, it reads the DOM (the structural code of the page) from top to bottom and converts each element into speech based on what that element is.

This means the order and type of your HTML tags directly control what the user hears and in what sequence. A `<div>` and a `<button>` may look identical on screen, but a screen reader announces them very differently — one is just a generic block; the other is announced as an interactive button that can be activated.

Why structure matters

Screen reader users commonly navigate a page in ways that don't involve reading everything line by line:

- By headings — jumping from `<h1>` to `<h2>` to `<h3>` to get a quick outline of the page, similar to scanning a table of contents
- By landmarks — jumping directly to `<nav>`, `<main>`, or `<footer>` sections

- By tab key — moving between interactive elements (links, buttons, form fields) in the order they appear in the HTML

If headings are out of order, or content isn't grouped using semantic tags, this navigation breaks down. The user has to listen to the entire page linearly instead of jumping to what they need — slow and frustrating.

This is why semantic HTML (**header**, **nav**, **main**, **section**, **footer**) isn't just good practice for organization — it's functional infrastructure for screen reader users.

2. Alt Text

Purpose

alt (alternative text) is an attribute on the `<img>` tag that provides a text description of an image. A screen reader reads this text aloud in place of the image. If an image fails to load, the browser also displays the alt text visually.

```

```

Writing good alt text

- Describe the content or purpose of the image, not just its appearance
- Keep it concise — usually one short sentence
- Do not start with “image of” or “picture of” — screen readers already announce that the element is an image
- Focus on what matters in context (a product photo on a shopping site needs different alt text than the same photo in a blog post about photography)

Decorative images

Some images exist purely for visual styling and carry no real information — a background flourish, a spacer graphic, a repeated decorative icon. These should use an empty alt attribute:

```

```

This tells the screen reader to skip the image entirely. This is different from leaving **alt** out completely — if **alt** is missing, some screen readers read the file name instead (e.g., “divider dot p n g”), which is confusing and useless to the listener.

Functional images

When an image acts as a button or a link (a search icon, a trash icon, a hamburger menu icon), the alt text should describe the action it performs, not the visual appearance of the icon.

Good:

```
<a href="/cart"></a>
```

Not this:

```
<a href="/cart"></a>
```

Complex images

Charts, graphs, and infographics carry more information than a single sentence can capture. Pair a short alt text summary with a longer description placed near the image or linked separately.

```

<p>Full data: sales rose from $2M in 2021 to $5.6M in 2024, with the
  steepest increase in 2023.</p>
```

3. Basic ARIA Roles

What ARIA is

ARIA stands for Accessible Rich Internet Applications. It's a set of HTML attributes that describe the role, state, and properties of an element to assistive technology — especially for elements that don't have built-in meaning.

The golden rule of ARIA

Use native HTML elements first. A `<button>` element is already keyboard-accessible and already announced correctly by screen readers — no ARIA needed. A `<div>` styled to look like a button has none of that built in. ARIA is meant to patch gaps in custom-built components, not replace elements that already exist for that purpose.

```
<!-- Good: native element, accessible by default -->
<button>Submit</button>

<!-- Needs extra work: div has no built-in role or keyboard behavior -->
<div role="button" tabindex="0">Submit</div>
```

If a native tag can do the job, use it. Reach for ARIA only when building something custom (like a tabbed interface or a modal popup) that HTML has no direct tag for.

Landmark roles

These identify the major regions of a page, letting screen reader users jump straight to a section instead of listening through everything above it.

Role	Purpose
banner	Site header
navigation	Navigation menu
main	Main content area
complementary	Sidebar or supporting content
contentinfo	Footer information
search	Search form

In practice, most of these are already provided automatically when you use the matching semantic tag (`<header>`, `<nav>`, `<main>`, `<aside>`, `<footer>`), so explicit roles are only needed when semantic tags aren't being used.

Widget roles

Used when building custom interactive components that HTML has no native tag for:

- `role="button"`
- `role="checkbox"`
- `role="tab"`
- `role="dialog"`
- `role="alert"`
- `role="menu"`

Common ARIA attributes

These attributes add extra information about state or labeling, and are often used alongside roles.

aria-label — accessible name when there's no visible text label

```
<button aria-label="Close menu">X</button>
```

aria-labelledby — points to another element's ID to use as the label

aria-describedby — points to an element that gives extra descriptive detail

aria-hidden="true" — hides purely decorative elements from screen readers

```
<span aria-hidden="true">★</span>
```

aria-expanded — tells the user whether a collapsible section is open or closed

```
<button aria-expanded="false">Show more</button>
```

aria-live — announces content that changes dynamically without a page reload

```
<div aria-live="polite">Your message was sent successfully.</div>
```

4. Example: Before and After

Before (inaccessible)

```
<div class="topbar">Site Menu</div>

<div onclick="openMenu()">≡</div>

```

After (accessible)

```
<nav aria-label="Main navigation">Site Menu</nav>

<button aria-label="Open menu" aria-expanded="false">≡</button>

```

What changed

- Generic `<div>` menu label replaced with a proper `<nav>` landmark
- Logo image given meaningful alt text
- Clickable icon `<div>` replaced with a real `<button>`, with `aria-label` describing its action and `aria-expanded` showing its state
- Purely decorative banner image given an empty `alt=""`

5. Key Takeaways

Remember for the exam

- Semantic HTML should always be the first choice — ARIA exists to fill gaps, not to replace correct tags
- Every image needs an `alt` attribute: a description for meaningful images, an empty value for decorative ones, and an action description for functional icons
- Screen readers depend entirely on structure — heading order, landmarks, and tab order determine how usable a page is for someone who cannot see it
- `aria-label`, `aria-hidden`, and `aria-expanded` add meaning to custom components, but incorrect or excessive ARIA can make a page LESS accessible than using no ARIA at all
- Accessibility is not a checklist item — it's a direct result of writing clean, structured, meaningful HTML